University Course Management System

Part 2, Task 3 — Reflection Questions: Solutions

These are model answers for the three Reflection Questions in `UnivCourseMgmt_Part2_Lab.docx`. Student answers don't need to match this wording — grade for whether they identified the same underlying idea, not for phrasing. Each answer below was checked against actual compiled code, not just reasoned about; see the verification note under Question 1.

Question 1

> ***Question:*** *The reference sketch labels the Course–Professor line 1–1 at both ends. Nothing in the Course or Department code actually stops two different Course objects from pointing at the same Professor*. Is the diagram wrong, or is it describing something the code should enforce but currently doesn't? What would you change (in the diagram, the code, or both) to make them agree?*

Model Answer

The diagram is wrong, not the code — and the fix belongs entirely on the diagram side, not the code side. Course really does have exactly one Professor* (that half of the 1–1 label, the one nearest Course, is accurate). But the other half — the "1" nearest Professor, which claims a Professor is associated with only one Course — is not true of the implementation and isn't something a full-credit answer should try to enforce in code, because it isn't true in the real world either: a professor ordinarily teaches more than one course. The correct multiplicity near Professor is n, not 1.

Nothing needs to change in `Course` or `Department` to fix this — a raw pointer is already a many-to-one-capable reference by default. Two different `Course` objects can each store the same `Professor*` value with no special support required, which is exactly what the diagram should have been showing all along. The takeaway for students: with aggregation via a raw pointer, **nothing** restricts how many owners can point at the same referenced object unless the class is deliberately written to check for that — sharing is the default, not the exception.

Verification: compiled and ran a minimal repro — two Course objects constructed with the same &professor argument — and confirmed it compiles cleanly and both Course objects report the identical Professor pointer at runtime. There is no language-level or design-level restriction preventing it.

```
Professor prof("Dr. Russell");
Course course1("Data Structures", &prof);
Course course2("Algorithms", &prof);   // reuses &prof - compiles and runs fine

// output:
// Data Structures -> Dr. Russell
// Algorithms -> Dr. Russell
// Same Professor object? yes
```

Full credit answer should reach the same conclusion by two different, equally valid paths: (a) relabel the diagram's Professor-side multiplicity from 1 to n, leaving the code untouched, or (b) argue that if the instructor truly wants a strict 1-to-1 rule enforced, that would require new code — e.g. Department or a course-scheduling layer tracking which Professors are already assigned and rejecting a second assignment — which is a much bigger change than the diagram implies, and arguably the wrong rule to build in the first place. Either path is acceptable; the point is recognizing the diagram currently overpromises a guarantee nothing enforces.

Question 2

> ***Question:*** *In real life a professor usually teaches more than one course. If you changed the Course–Professor multiplicity to n–1 (many Courses, one Professor each, but a Professor can appear on more than one Course), would anything in your Course or Department implementation need to change, or does the existing Professor* design already support that?*

Model Answer

Nothing needs to change. This follows directly from the Question 1 verification: `Professor* professor` inside `Course` is just a reference to wherever the caller points it — the class itself has no idea, and no way to know, whether it's the only Course pointing at that Professor or one of ten. The n–1 relationship the question describes isn't a new feature to build; it's the relationship the code already has, and the 1–1 label in the original sketch was the part that didn't match reality.

This is worth calling out explicitly to students: aggregation via a raw (or smart, non-owning) pointer naturally supports many-to-one and many-to-many sharing with zero extra code, because the referenced object's identity and lifetime are completely independent of how many places hold a pointer to it. Composition (storing by value, as Department does with its vector<Course>) does not have this property — a value member belongs to exactly one container by construction, which is precisely why composition was the right choice for Department–Course and would be the wrong choice for Course–Professor.

Question 3

> ***Question:*** *Composition is drawn with a filled diamond only between Department and Course, never between Course and Student, even though a Course "has" Students too. In one or two sentences, say why a Course's relationship to its Students is aggregation and not composition — think about what happens to a Student object when the Course they're enrolled in is destroyed.*

Model Answer

A Student's existence doesn't depend on any one Course: the same `Student` object is typically enrolled in several Courses at once, and continues to exist — with other code still holding valid pointers to it — even after a particular Course object is destroyed or the student drops it. If Course destruction deleted its enrolled Students the way Department's destruction is allowed to let its Courses go, every other Course that student was enrolled in would be left holding a dangling pointer. That's the defining test for aggregation vs. composition: does the *part* have a legitimate reason to outlive this particular *whole*? For a Student and one of their Courses, yes — so the relationship is aggregation (open diamond), never composition.

A useful contrast for students who want the fuller picture: this is exactly symmetric with why Course–Professor is aggregation too (Question 1/2), and exactly the opposite of why Department–Course is composition — a Course object in this design has no purpose or identity outside belonging to the one Department that created it, so it's correct and safe for it to be destroyed alongside that Department.